Software Architecture & Design Specification

for

Bank Management System

Version 1.2

Prepared by:

Abhishek Hanamant Kinagi (PES1UG24CS018), Adithya Ajeeth (PES1UG24CS022), Aditya M (PES1UG24CS027), Aditya Vats (PES1UG24CS032)

PES University — Software Engineering (UE24CS341A)

Reference: Software Requirements Specification, Bank Management System, v1.2

Revision History

Version	Date	Changes
1.0	27 Sep 2026	First version.
1.1	26 Oct 2026	S-36: traceability, API table and data design updated to match the build.
1.2	7 Nov 2026	S-46: final API table and error codes, dashboard component, STRIDE threat summary, open issues and next steps, the SRS v1.2 requirement IDs (FR-15 to FR-17, SEC-5 and SEC-6), and the sections from the course SAD template: approvals, audience, goals and stakeholders, technology stack, risks, logging and monitoring, UX design and appendices.

Approvals

Role	Name	Signature/Date
Lead - Platform, Security & Integration	Aditya Vats (PES1UG24CS032)	Approved, 7 Nov 2026
Identity, Administration & UI	Abhishek Hanamant Kinagi (PES1UG24CS018)	Approved, 7 Nov 2026
Customers & Accounts	Adithya Ajeeth (PES1UG24CS022)	Approved, 7 Nov 2026
Transactions & Reporting	Aditya M (PES1UG24CS027)	Approved, 7 Nov 2026

1. Introduction

1.1 Purpose

This Software Architecture and Design Specification (SADS) describes the architecture and detailed design of Version 1.0 of the Bank Management System. It translates the functional and non-functional requirements defined in the Software Requirements Specification (SRS) into a concrete architectural structure, component decomposition, and detailed design artifacts, providing the development team with a blueprint for implementation.

1.2 Scope

This document covers the architecture and design of the complete Bank Management System as scoped in the SRS: a browser-based presentation layer, an API layer, a C/C++ back-end core implementing all business logic, and a relational database. It includes the architectural pattern, component decomposition, security architecture, sequence-level interaction design, API design, and error-handling strategy. Design decisions for out-of-scope features (Appendix D of the SRS) are not addressed.

1.3 Definitions, Acronyms and Abbreviations

Term	Description
RBAC	Role-Based Access Control

API	Application Programming Interface
RDBMS	Relational Database Management System
FR / NFR	Functional Requirement / Non-Functional Requirement
ACID	Atomicity, Consistency, Isolation, Durability (transactional properties)
DTO	Data Transfer Object — structured payload exchanged across the API layer
Audit Trail	Append-only record of every balance-changing operation and its operator

1.4 References

- Software Requirements Specification (SRS) — Bank Management System, Version 1.2.
- IEEE Std. 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- IEEE Std. 1016-2009, IEEE Standard for Information Technology — Systems Design — Software Design Descriptions.

1.5 Overview

Section 2 presents the architectural design, including the chosen architectural pattern, the component diagram, security architecture, traceability to requirements, goals and stakeholder concerns, the technology stack, and risks. Section 3 presents the detailed design, including sequence diagrams for the two most critical transaction flows, the API design, the error-handling strategy, data design and UX design. Section 4 documents design constraints and their rationale. Section 5 lists the open issues and next steps, and Section 6 holds the appendices. Section 2.4.3 summarizes the threats to the system.

1.6 Intended Audience

This document is for the project team, the course instructor and viva evaluators, and any developer, tester or security reviewer who maintains or checks the system.

2. Architectural Design

2.1 Architectural Pattern

The system adopts a layered (n-tier) architecture consisting of four logical layers: Presentation, API/Gateway, Business Logic (C/C++ core), and Data. This pattern was selected, consistent with SRS Section 2.5, because it:

- Cleanly separates presentation concerns from banking business rules, ensuring the front end contains no banking logic (SRS 2.5) and that the same rules apply regardless of client.
- Allows the C/C++ back-end core to be developed, tested, and evolved independently of the front end (supports NFR-8 Maintainability).
- Centralizes RBAC enforcement in a single layer rather than duplicating it across clients (supports NFR-4 Security).
- Is well suited to a small, four-person academic team, as each layer can be owned by a subset of the team with minimal cross-layer coupling.

An alternative microservices-based decomposition was considered and rejected: given the project's scope (single branch, single currency, three roles) and timeframe, the operational overhead of independently deployed services would outweigh any benefit, and would conflict with the requirement that a single C/C++ core owns all transaction logic.

2.2 Component Diagram

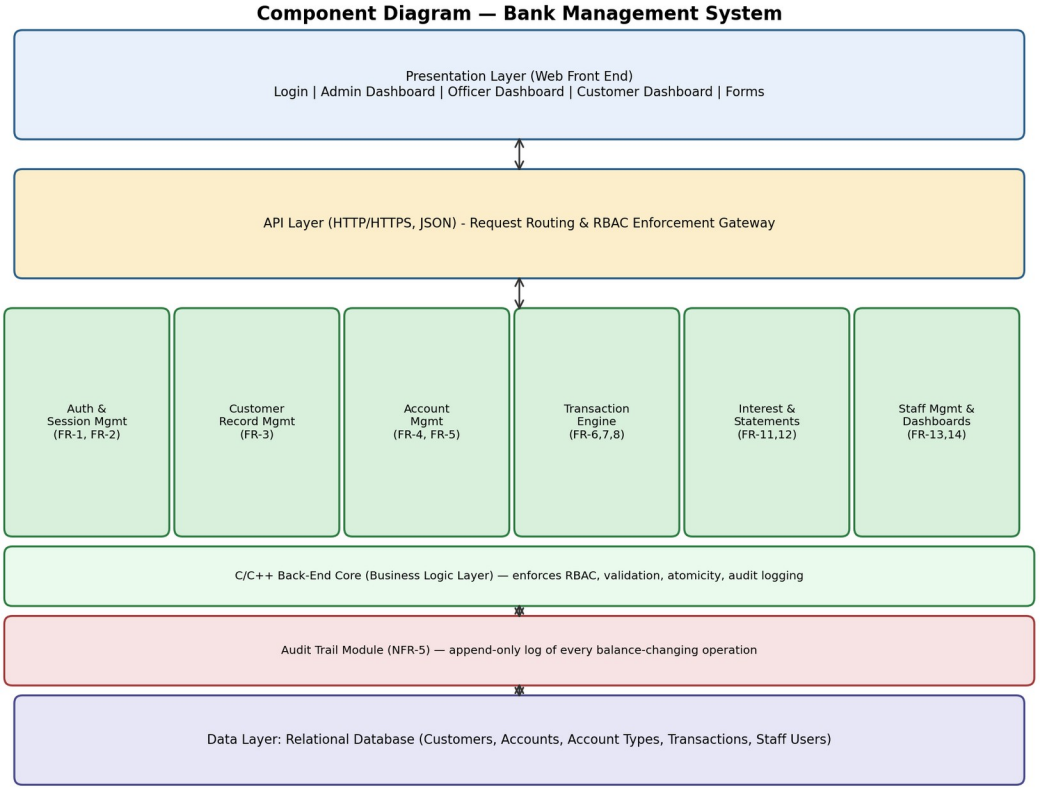


Figure 1: Component Diagram — Bank Management System

2.2.1 Component Description

Component	Responsibility
Presentation Layer	Renders the role-specific dashboards (frontend/landing.js) and forms; performs client-side input formatting only; contains no balance-affecting logic.
API Layer / Gateway	Exposes REST endpoints over HTTP on the loopback address, or over HTTPS when a certificate is configured; authenticates requests; performs the first-line RBAC check before delegating to the core; serializes/deserializes JSON.
Auth & Session Mgmt	Validates credentials, manages sessions, enforces the 3-strikes account lockout rule, issues session tokens; logout invalidates only the current token. Lists locked users and lets an Administrator unlock them (FR-16).
Customer Record Mgmt	CRUD operations on customer master records; enforces deactivation rules.
Account Mgmt	Opens, freezes, unfreezes, and closes accounts; enforces minimum-balance-at-opening and zero-balance-at-closure rules. Lets an Administrator create and change account types and their parameters, and lets staff list them (FR-15).
Transaction Engine	Processes deposits, withdrawals, and transfers as atomic units of work; enforces minimum balance and daily withdrawal limit rules.
Interest & Statements	Serves balance inquiry and transaction history (FR-9, FR-10), where a Customer sees only the accounts they hold and a Bank Officer or Administrator any account; computes and credits periodic interest; generates account statements for a selected date range.
Staff Mgmt & Dashboards	Administrator management of Bank Officer accounts. Builds the role-specific dashboard (GET /dashboard): customer, account and staff counts and today's transactions for an Administrator; the transactions a Bank Officer processed

	today; and a Customer's own accounts and balances.
Audit Trail Module	Keeps an append-only record of every balance-changing operation; not editable or deletable by any role. Administrators browse it read-only through GET /audit and frontend/audit.js (FR-17).
Data Layer (RDBMS)	Persists all entities; provides transactional guarantees (ACID) that the Transaction Engine relies on for atomicity.

2.3 Traceability Matrix (Architecture to Requirements)

Component	Requirements Traced
Auth & Session Mgmt	FR-1, FR-2, FR-16, NFR-4, SEC-1, SEC-2, SEC-5, SEC-6
Customer Record Mgmt	FR-3
Account Mgmt	FR-4, FR-5, FR-15
Transaction Engine	FR-6, FR-7, FR-8, NFR-1, NFR-3, NFR-7
Interest & Statements	FR-9, FR-10, FR-11, FR-12, NFR-2
Staff Mgmt & Dashboards	FR-13, FR-14
Audit Trail Module	NFR-5, FR-17, SEC-4
API Layer / Gateway	FR-1 (RBAC enforcement), NFR-4, NFR-7 (input validation and clear errors), SEC-1, SEC-3
Data Layer (RDBMS)	NFR-3 (atomicity), Section 6 (RDBMS requirement)
Presentation Layer	FR-14 and FR-17 (dashboard and audit screens), NFR-2 (page load time), NFR-6 (usability), NFR-9 (Chrome and Firefox without browser-specific code)
All back-end components	NFR-8 (one module per functional area), NFR-9 (Linux and Windows builds in CI, no platform-specific business logic)

2.4 Security Architecture

2.4.1 Security Objectives

- Ensure that every request reaching the business logic layer is authenticated and authorized according to the requester's role, regardless of which client issued it (supports FR-1, NFR-4).
- Protect credentials at rest with salted hashes, and protect credentials and financial data in transit with HTTPS on network deployments (supports NFR-4, SEC-2, SEC-3).
- Preserve an append-only record of every balance-changing action for accountability (supports NFR-5, SEC-4). This protects the record within the application; it is not cryptographic tamper evidence.

2.4.2 Security Requirements (Architecture-Level)

ID	Requirement
SEC-1	The API Layer shall reject any request lacking a valid session token, and the Business Logic Layer shall independently re-verify the caller's role for every operation, so that RBAC is never enforced by the front end alone (traces to FR-1, NFR-4).
SEC-2	Passwords shall be hashed with salted Argon2id before persistence. Plaintext credentials shall not be logged, and session tokens shall not be persisted in the browser. The only exception to showing a credential is the temporary password returned once when a customer or staff user is created, for handoff (traces to NFR-4).
SEC-3	All client-server communication in a network-accessible deployment shall use HTTPS/TLS to protect credentials, account numbers, and balances in transit (traces to SRS 3.4). Plain HTTP is

	allowed only on the loopback address, for development and demos with fictional data; the default build does not enable HTTPS.
SEC-4	Every balance-changing operation shall generate an append-only audit record (timestamp, accounts affected, amount, operator ID) that cannot be modified or deleted through the application, and the audit viewer shall be restricted to Administrators (traces to NFR-5, FR-17).
SEC-5	The system shall lock an active user on the third consecutive failed sign-in, and only an Administrator shall be able to unlock that user (traces to FR-2, FR-16).
SEC-6	Session tokens shall be random 64-character values with a fixed 30-minute expiry. Logout shall invalidate the current token, and requests from inactive or locked users shall be rejected (traces to FR-2).

2.4.3 STRIDE Threat Summary

Threat	Main mitigations	Remaining risk
Spoofing: stolen or guessed credentials	Passwords hashed with Argon2id; lockout after three failed sign-ins; random session tokens that expire after 30 minutes; first sign-in forces a password change.	No second factor. A stolen token works until it expires or the user signs out.
Tampering: changing balances or records outside the rules	All rules run in the server core, not the browser; each balance change is one atomic database transaction; triggers block UPDATE and DELETE on the transactions and audit_log tables.	Someone with direct access to the database file can remove the triggers.
Repudiation: denying an action	Audit trail records every balance change and security event with the operator ID and time; refused requests are recorded as ACCESS_DENIED.	The audit trail is protected inside the database only; it is not signed or copied elsewhere.
Information disclosure: reading other people's data	Customers can reach only their own records, and another customer's account looks the same as a missing one; HTTPS support, with sign-in and customer data refused over plain HTTP on network addresses; secrets are kept out of logs.	The SQLite file is not encrypted at rest, so it relies on operating system file permissions.
Denial of service: making the system unavailable	Request body size limit (413 PAYLOAD_TOO_LARGE); no error exposes internal details; failed operations roll back cleanly.	No per-client rate limit, and anyone who knows a username can lock that account with failed sign-ins.
Elevation of privilege: acting above your role	The role comes from the server-side session and is checked in the core for every operation; hiding screens in the browser is only a convenience.	Administrator actions rely on one password, with no second approval.

2.5 Deployment View

The system is deployed as a client-server configuration: any modern web browser (client) communicates with a server hosting the API layer and C/C++ business logic core, over HTTP on the loopback address or

over HTTPS on a network address. The core uses an embedded SQLite database file on the same machine. All of this may run on one machine for development and testing; a network deployment needs a certificate and key (Section 5).

2.6 Goals and Stakeholder Concerns

The main architectural goals are that every balance change is atomic and audited, that access is enforced by role in the server core, that the code stays modular and builds on Linux and Windows, and that the system runs as one server process with one database file.

Stakeholder	Main concerns
Customers	Correct balances; other customers cannot see their data.
Bank Officers	Quick, accurate transaction processing and clear reasons when a request is refused.
Administrators	Control of staff, account types and locked users; a read-only view of the audit trail.
Course evaluators	Traceability from requirements to design and tests; documented deviations.
Development team	One module per functional area, automated tests and CI on both platforms.

2.7 Technology Stack and Data Stores

Concern	Choice
Back-end language and build	C++17 with CMake
HTTP server	cpp-httplib 0.57.1
JSON	nlohmann/json 3.12.0
Data store	SQLite 3.53.3, embedded, in one database file
Password hashing	libsodium (Argon2id)
Front end	Plain HTML, CSS and JavaScript, with no framework
Testing and coverage	GoogleTest and CTest; gcovr for line and branch coverage
Continuous integration	GitHub Actions on Ubuntu and Windows

The reasons for these choices are in docs/decisions/001-tech-stack.md (ADR 001).

2.8 Risks and Mitigations

Risk	Mitigation
Lost updates or partial balance changes under concurrent requests	Each balance change is one BEGIN IMMEDIATE database transaction, so writes run one at a time and roll back as a whole (deviations 1 and 22).
Banking or role rules bypassed by calling the API directly	All rules and role checks run in the server core; the browser only hides screens (Section 2.4).
Stolen or guessed credentials	Argon2id hashes, lockout after three failed sign-ins and 30-minute session tokens (Section 2.4.3).
Code that works on only one platform	CI builds and tests on Ubuntu and Windows, and scripts/check-portability.sh rejects platform-specific code in the business modules.
The provisional interest basis needs to change	The basis is recorded in ADR 003 and deviation 6, so it can be confirmed or changed explicitly.
Loss of the single database file	The system has no built-in backup; a real deployment must back up the file. This is outside the project's scope.

3. Detailed Design

3.1 Sequence Diagrams

3.1.1 Sequence Diagram 1: Deposit / Withdrawal Transaction

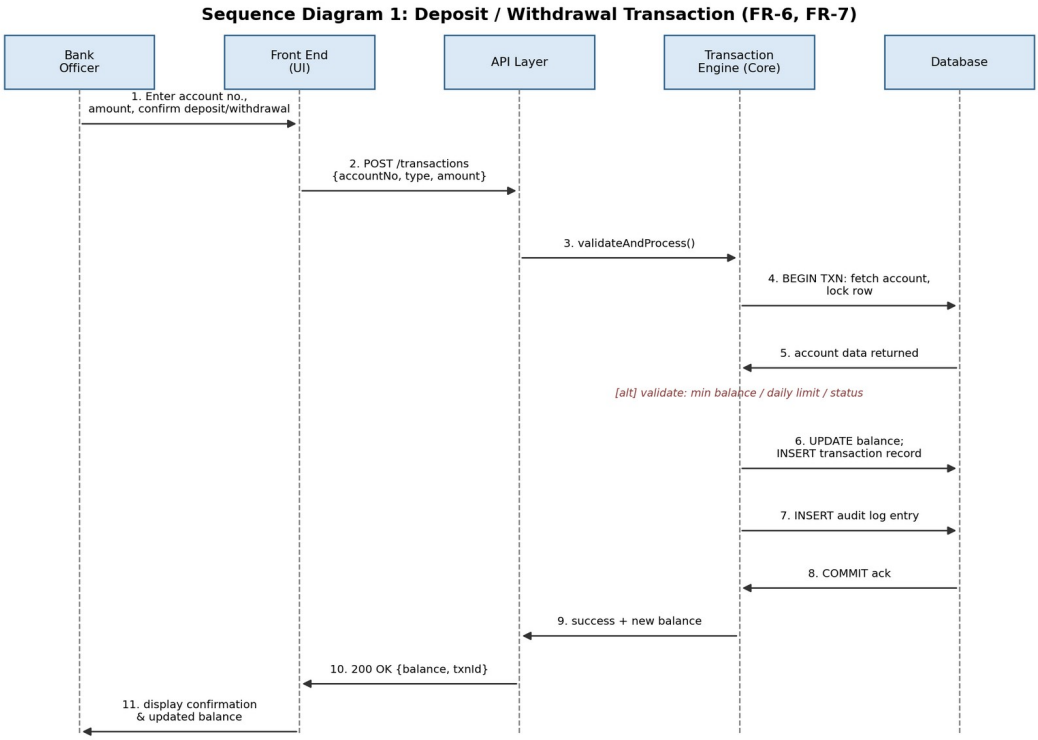


Figure 2: Sequence Diagram — Deposit / Withdrawal Transaction (traces to FR-6, FR-7, NFR-1, NFR-3)

This flow shows a Bank Officer initiating a deposit or withdrawal. The Transaction Engine takes the database write lock, reads the account, validates the operation (minimum balance for withdrawals, daily withdrawal limit, account status), applies the balance update and transaction record within a single database transaction, writes an audit log entry, and commits — satisfying the atomicity requirement (NFR-3) and the 2-second performance target (NFR-1).

3.1.2 Sequence Diagram 2: Fund Transfer

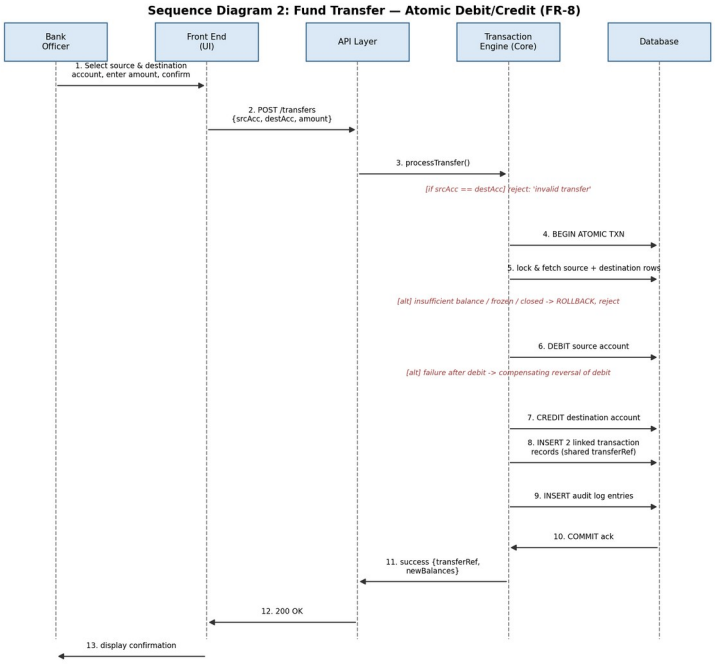


Figure 3: Sequence Diagram — Fund Transfer, Atomic Debit/Credit (traces to FR-8, NFR-3)

This flow shows a fund transfer between two accounts held at the bank. The Transaction Engine rejects same-account transfers immediately, then, holding the database write lock, reads both accounts and validates balance/status on both sides, debits the source, credits the destination, and inserts two linked transaction records sharing a common transfer reference — all within one atomic unit of work. If a failure occurs after the debit but before the credit, the whole database transaction is rolled back, ensuring neither account is left in a partially updated state.

3.2 API Design

The API layer exposes REST-style endpoints over HTTP/HTTPS, exchanging JSON payloads. The table below summarizes the primary endpoints; all endpoints require a valid session token and are subject to RBAC enforcement in the business logic layer. Role checks are made in the back-end core as well as in the API layer; a refused request returns 403 and is recorded in the audit trail. No endpoint edits or deletes an audit entry: POST, PUT, PATCH and DELETE on /audit return 405. The table lists every endpoint in the final build. POST /auth/login and GET /health need no session token. Over plain HTTP on a network-accessible address, sign-in and customer data are refused; HTTPS or the loopback address is required.

Method & Path	Description	Required Role
GET /health	Report server status and version	None
POST /auth/login	Authenticate user, issue session token	Any
POST /auth/logout	Terminate active session	Any (authenticated)
POST /auth/change-password	Set a new password on first sign-in	Any (authenticated)
GET /users/locked	List users locked after three failed sign-ins	Admin
PATCH /users/{id}/unlock	Unlock a locked user (customer or staff) and reset the failure count	Admin
POST /customers	Create a new customer record	Officer, Admin
GET /customers	Search customer records by customer	Officer, Admin

	ID, name or phone number	
GET /customers/me	Retrieve the signed-in customer's own record	Customer
GET /customers/me/accounts	List the signed-in customer's own accounts, with type, status and balance	Customer
GET /customers/{id}	Retrieve a customer record, including its accounts	Officer, Admin, Customer (own)
PUT /customers/{id}	Update a customer record; a body with only status: inactive deactivates the customer	Officer, Admin to update; Admin to deactivate
POST /accounts	Open a new account for a customer	Officer, Admin
PATCH /accounts/{acc}/status	Freeze, unfreeze or close an account	Officer, Admin
GET /account-types	List account types and their parameters	Officer, Admin
POST /account-types, PUT /account-types/{id}	Create or change an account type and its parameters	Admin
POST /transactions	Process a deposit or withdrawal	Officer, Admin
POST /transfers	Process a fund transfer between two accounts	Officer, Admin
GET /accounts/{acc}/balance	Retrieve current balance	Officer, Admin, Customer (own)
GET /accounts/{acc}/transactions	Retrieve transaction history (filterable by date)	Officer, Admin, Customer (own)
GET /accounts/{acc}/statement	Generate a statement for a date range	Officer, Admin, Customer (own)
POST /interest/run	Credit interest for a completed calendar month	Admin
POST /staff	Create a Bank Officer account	Admin
GET /staff	List Bank Officer accounts	Admin
PATCH /staff/{id}	Rename, deactivate or reactivate a Bank Officer account	Admin
GET /audit	Browse the audit trail, newest first, filtered by date range and account; read-only	Admin
GET /dashboard	Retrieve role-specific dashboard summary	Any (authenticated)

3.3 Error Handling

The system applies a consistent error-handling strategy across all layers:

- Input validation (empty fields, non-numeric or negative amounts, malformed account numbers) is performed at the API layer before any request reaches the business logic core, and re-validated within the core itself, so that no invalid input can modify a stored balance or record (supports NFR-7).
- Business rule violations (insufficient balance, minimum balance breach, daily limit exceeded, frozen/closed account, same-account transfer) are returned as structured error responses with an error code and human-readable reason, leaving all balances unchanged.

- Any failure during a multi-step balance-changing operation (e.g., a crash between debit and credit in a transfer) rolls back the single database transaction that holds the whole operation, so the account is never left in a partially updated state (supports NFR-3).
- Unexpected server-side errors are caught, logged internally (without exposing internal details to the client), and surfaced to the user as a generic failure message with guidance to retry or contact support.
- Concurrency conflicts (e.g., two operations on the same account) are handled by the database write lock. The chosen RDBMS, SQLite (ADR 001), has no row-level locking, so every balance-changing operation starts with BEGIN IMMEDIATE and writes run one at a time; the second request then reads the updated balance and either succeeds or is rejected with a clear message, and no update is lost. The row locks drawn in Figures 2 and 3 are realised by this database-wide lock. Record-level locking is future work (SRS Appendix D); see the deviations log.

Error Category	Example	System Response
Validation error	Negative or non-numeric amount	Reject before processing; no balance change; field-level error message
Business rule violation	Withdrawal breaches minimum balance	Reject; state specific reason; no balance change
Authorization failure	Customer attempts to view another customer's account	Deny request; log the attempt
Concurrency conflict	Two simultaneous withdrawals on the same account	Database write lock serializes operations; second request processed against updated balance or rejected if now invalid
Partial failure	Crash between debit and credit in a transfer	Automatic rollback; account restored to prior state

Logging and monitoring: the server writes timestamped log lines to standard output and standard error through one logging module, which must never receive passwords, session tokens or request bodies. GET /health reports the server status and version, and the audit trail records every balance change and security event. There is no separate metrics or alerting system.

3.3.1 Error Codes

Errors are returned as JSON with a stable code and a readable message, plus field details where they apply. Each code maps to one HTTP status.

HTTP status	Codes	Meaning
400	VALIDATION_ERROR, INVALID_JSON, BAD_REQUEST	Bad input; nothing is changed
401	UNAUTHENTICATED, ACCOUNT_LOCKED	No valid session, or sign-in refused
403	FORBIDDEN	The role or ownership check failed, or plain HTTP was refused
404	NOT_FOUND	The record does not exist or is not visible to this user
405	METHOD_NOT_ALLOWED	The method is not allowed, for example changing the audit trail
409	ACCOUNT_BALANCE_NONZERO, ACCOUNT_STATUS_CONFLICT,	The request clashes with the current state

	CUSTOMER_ID_EXHAUSTED, CUSTOMER_INACTIVE, DUPLICATE_ACCOUNT_TYPE_NAME, DUPLICATE_CUSTOMER_FIELD, DUPLICATE_USERNAME, INTEREST_ALREADY_RUN, PASSWORD_CHANGE_NOT_REQUIRED, USER_NOT_LOCKED	
413	PAYLOAD_TOO_LARGE	The request body is over the size limit
422	ACCOUNT_NOT_ACTIVE, BALANCE_LIMIT_EXCEEDED, BELOW_MINIMUM_BALANCE, DAILY_LIMIT_EXCEEDED, INSUFFICIENT_BALANCE	Valid input that breaks a bank rule; balances are unchanged
500	INTERNAL_ERROR	Unexpected failure; details are logged, not returned

3.4 Data Design

The data layer uses a relational schema consistent with SRS Appendix B: Customer (1:N) Account, Account Type (1:N) Account, Account (1:N) Transaction, and Staff User (1:N) Transaction. A customer has zero or one linked login. Foreign keys enforce these relationships, and database-level transactions provide the atomicity guarantees required by NFR-3. Transfer pairs are linked logically through transfer_ref and interest entries through interest_run_id; neither has a declared foreign key. Full attribute lists, keys and constraints are in db/schema.sql, and the diagrams are in the repository's design/ directory.

The entity-relationship diagram of the implemented schema is Figure 4 and is kept in the repository as design/er-diagram.png (Mermaid source design/er-diagram.mmd, SVG design/er-diagram.svg). It adds the users table (role, status, failed-attempt count), the append-only audit_log table, the interest_runs table and the account type parameters to the entities listed above; audit_log and transactions are made append-only by database triggers. Figure 4 predates SRS v1.2 Appendix B.2, which corrects two relationships: the customer login is optional and interest_run_id has no declared foreign key. db/schema.sql is authoritative.

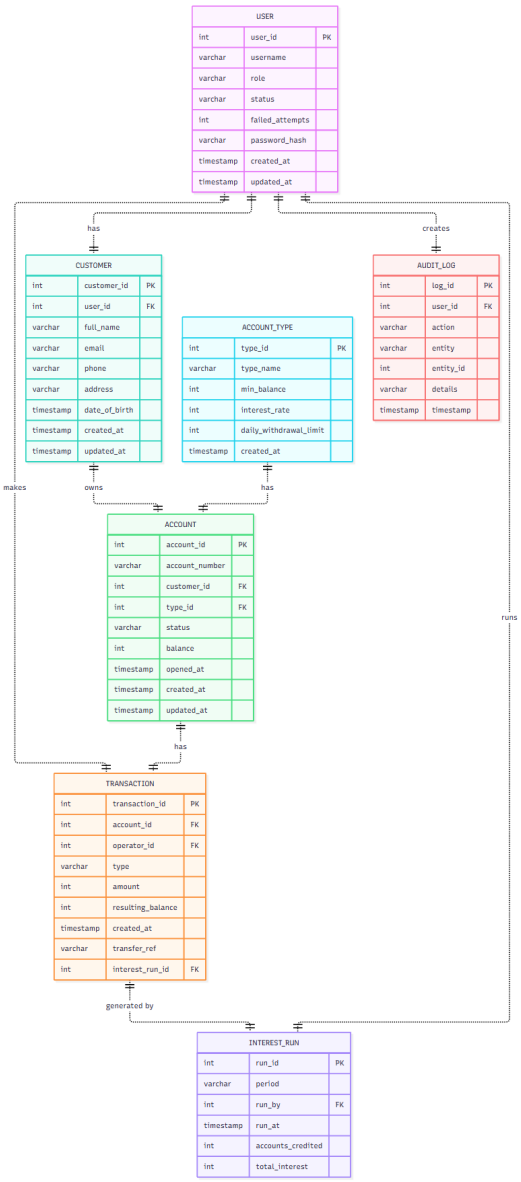


Figure 4: Entity-Relationship Diagram — Bank Management System database

3.5 UX Design

The front end is one web page whose sections are shown according to the signed-in role, starting from that role's dashboard. Each balance-changing action asks for confirmation before anything is sent and then shows the result, including the new balance. Forms show field-level errors, password fields are masked, and status messages use ARIA live regions so that screen readers announce them. Usability for first-time users (NFR-6) is not yet validated: see Section 5.

4. Design Constraints and Rationale

Constraint	Rationale
Business logic implemented in C/C++	Mandated by the assigned project topic (SRS 2.5); centralizing logic in one core also enforces uniform rule application regardless of client.
Front end contains no banking logic	Ensures RBAC and validation cannot be bypassed by manipulating the client (SRS 2.5, NFR-4).

Relational database required	Referential integrity between customers, accounts, and transactions, and ACID guarantees for balance updates (SRS 2.5, Section 6).
Three fixed roles only	Scoped to the academic project's requirements; simplifies the RBAC model (SRS 2.5).
Layered architecture	Supports maintainability and independent testing by a four-person team (SRS 2.5, NFR-8).
Git/GitHub-based development	Required by the course for collaborative, version-controlled development (SRS 2.5, Section 6).

5. Open Issues and Next Steps

Open issue	Effect	Next step
HTTPS needs a certificate at deployment	Without one, the server refuses sign-in and customer data over plain HTTP on network addresses, so it only works on the loopback address.	Provide a certificate and key for any real deployment (README, HTTPS and secrets).
Sessions are held in server memory	A restart signs everyone out, and the server cannot run as several processes.	Move sessions to the database if more than one server process is ever needed.
SQLite allows one writer at a time	Balance changes run one after another. This meets the current targets (deviation 1).	Move to a client-server RDBMS if concurrent load grows.
Database file is not encrypted at rest	Passwords are hashed, but customer details and balances rely on file permissions.	Use disk or database encryption in a real deployment.
No per-client rate limit	Failed sign-ins can lock an account on purpose, and an Administrator has to unlock it.	Add rate limiting in front of the sign-in endpoint.
Usability is not validated	TC-21, the usability study with three first-time users, was not run and is recorded as deviation 35, so NFR-6 is not validated and the System Validation Report's exit criteria are not met. Every other planned case passed, including TC-22 after the mobile table fix.	Run TC-21 with three first-time users, using the protocol in docs/USABILITY_WALKTHROUGH.md, and update docs/System_Validation_Report.md.
Differences from the SRS and earlier SADS versions	Recorded with reasons in docs/DEVIATIONS.md.	Keep the log current; review it for each release.

6. Appendices

Glossary: see Section 1.3. References: see Section 1.4.

Tools: Mermaid for the ER diagram source, LibreOffice for the PDF export, gcovr for coverage, puppeteer-core for the browser checks, and GitHub Issues, Projects and Actions for the backlog and CI.